Red Black Trees

* Node Color (Red/Black)
* Root Property (Root always Black)
* Red Node Property (always Black childs)
* Leaf Property (Always Black) i.e nullptr is black.
* Depth Property (Right & left path has same number of Black nodes)

→ if you start to traverse from any any node, you found the same number of Black nodes either in right direction path or left direction path.

* Leaf node can be red but the nullptr as black.

→ * Path not means right subtree whole or left subtree whole.

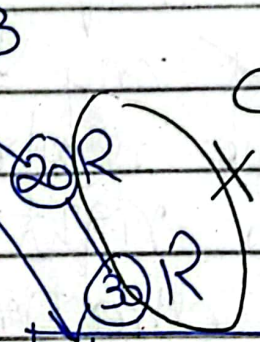
Insertion

10, 20, 30, 15, 25, 5, 12, 35, 40, 32, 50, 11

1) Insert the new node with color red
2) if node is root, recolor it black
3) Red-Red Violation

* Case 1: Uncle is Red

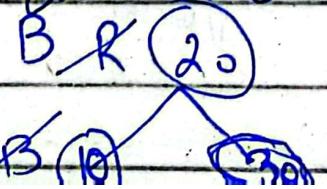
→ Recolor the ~~new node~~ parent and uncle to black and the grandparent to red. Then, repeat the fix-up process from the grandparent (if gp is not root)



* Case 2: Uncle is Black / Null

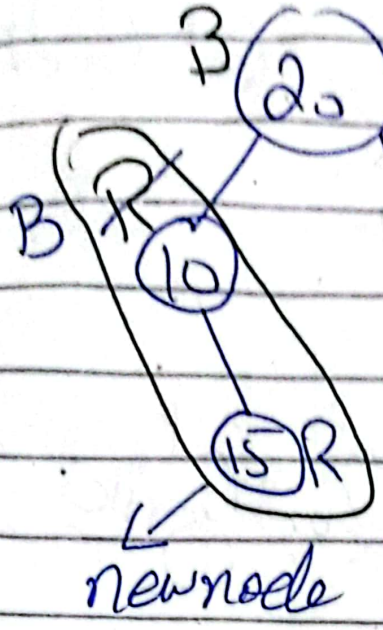
→ perform Rotations to balance the tree.

→ Recolor the nodes accordingly.

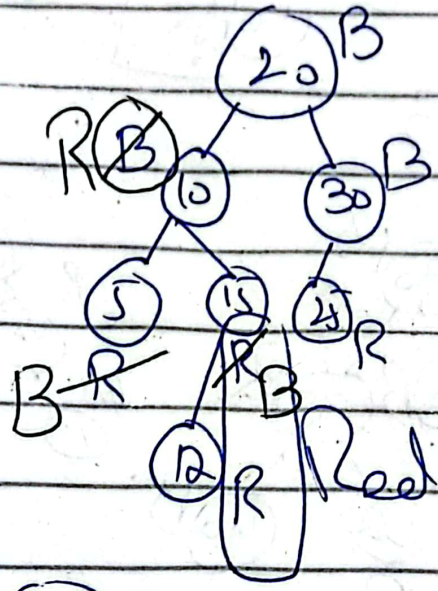


or swap

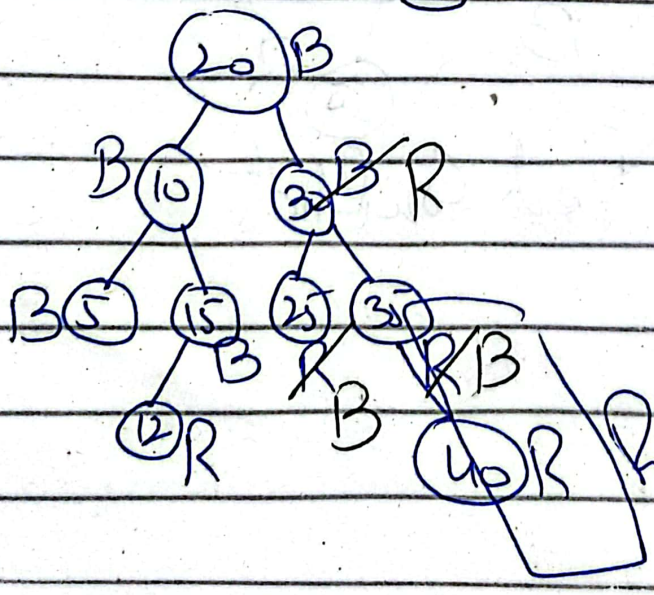
only Recolor the nodes that you rotates (i.e nodes that changes its positions)



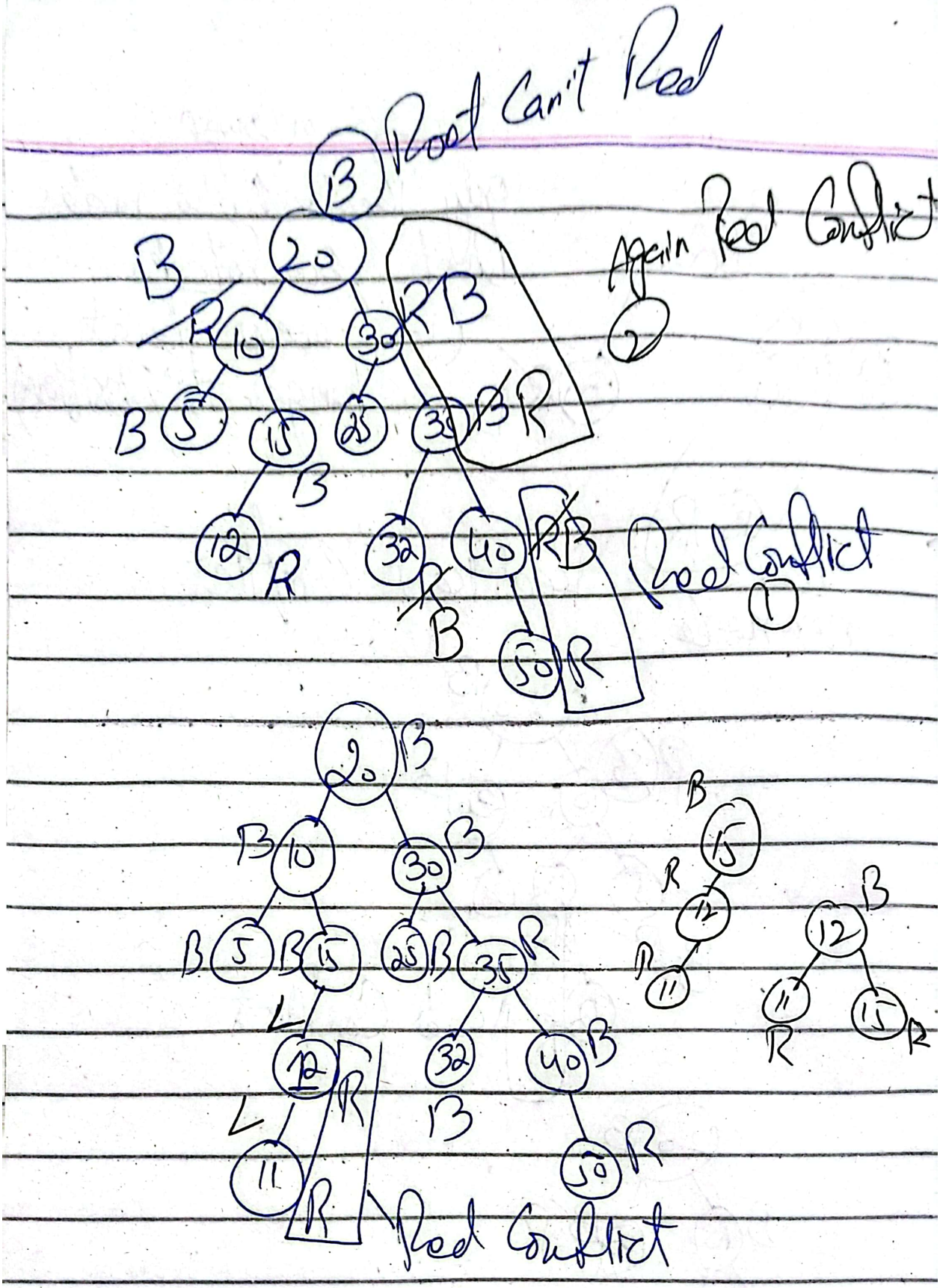
Red-Red Conflict

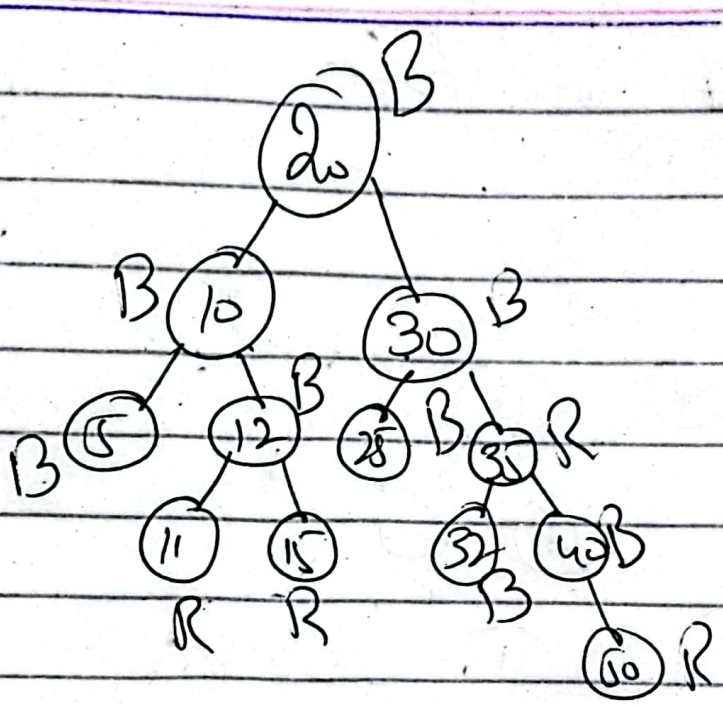


Red Conflict

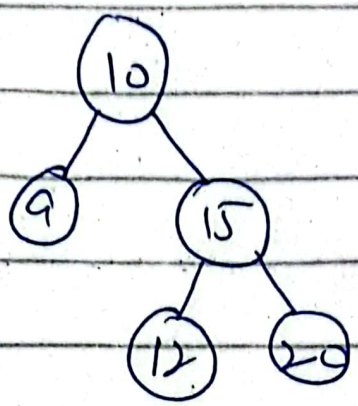


Red Conflict


Root Can't Red
Again Red Conflict
Red Conflict
Red Conflict


20
B
10
B
30
B
5
B
12
B
25
B
35
R
11
R
15
R
32
B
40
B
50
R

Deletion in Red Black Trees



First in BST deletion, We can use 2 approaches

* inorder predecessor
* inorder successor

largest element from the left subtree of that node

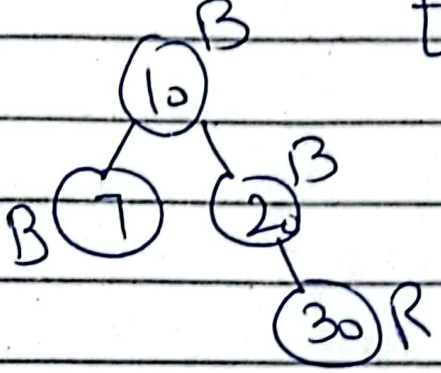
→ smallest element from the right subtree of that node

Step 1:-

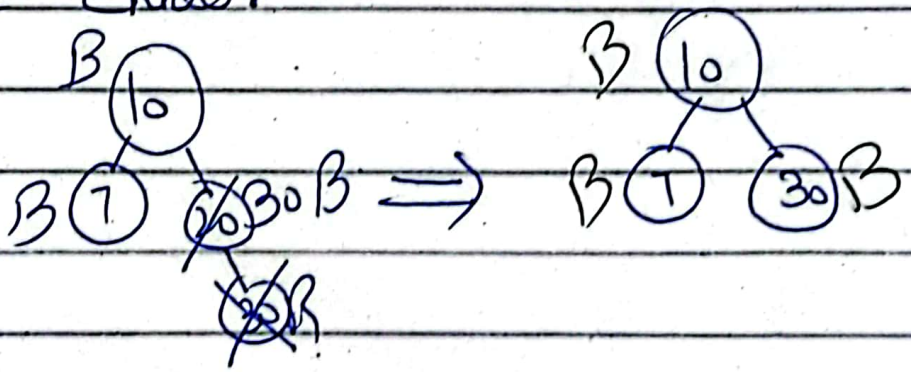
Perform BST deletion.

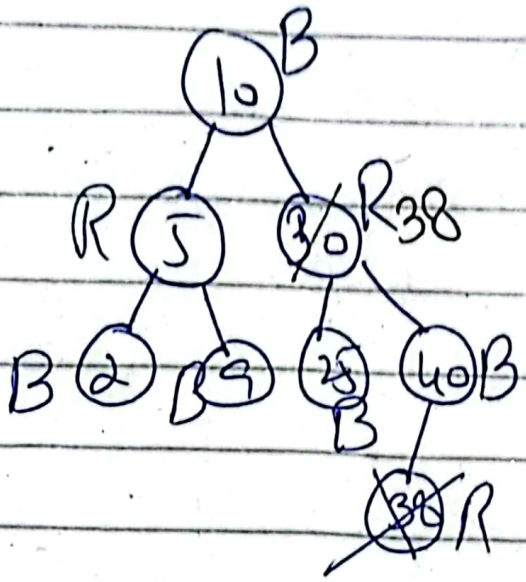
Step 2:-

* Case 1:- If node to be deleted as Leaf node as well as red, just delete it. like if we want to delete 30.

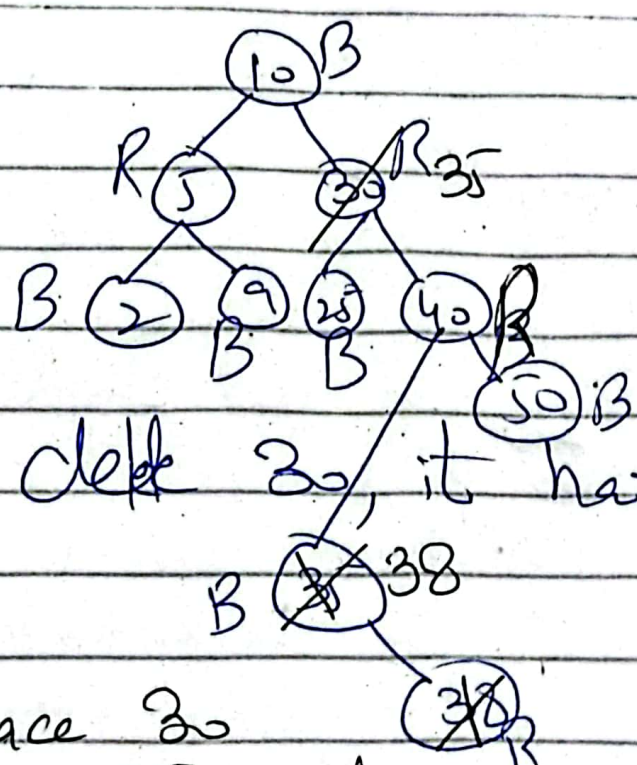


If we want to delete 20, it has one child.





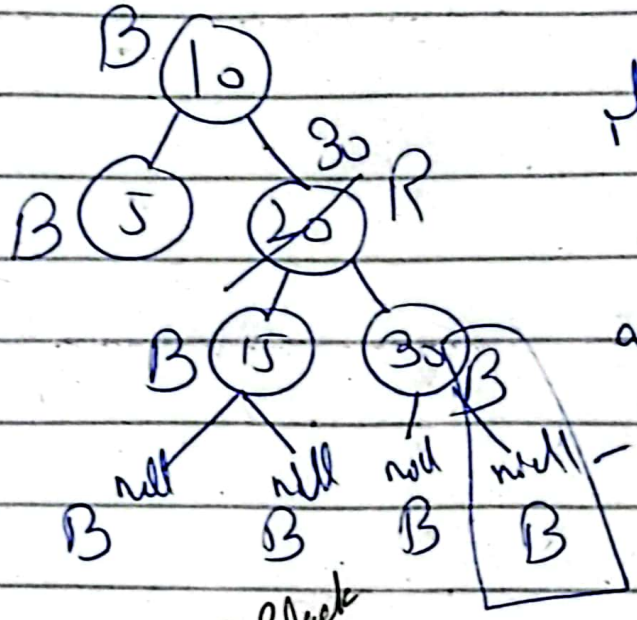
38 is leaf node, simply delete it as, it is red.



If delete 30, it has two childs.

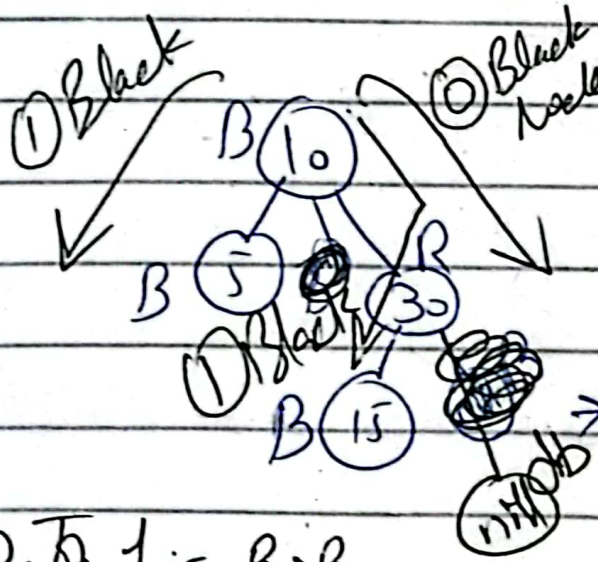
Replace 30 with 35 and 35 with 38 to delete only red node with no childs

Case 2:- To delete black node and that black node contains black childrens.



It will delete 20, just replace it with 30 and delete 30

- Double Black (DB) Conflict



As you see it voidates the property of red-black trees from root, it should have same number of black nodes either from ~~right subtree or left subtree~~ any path.

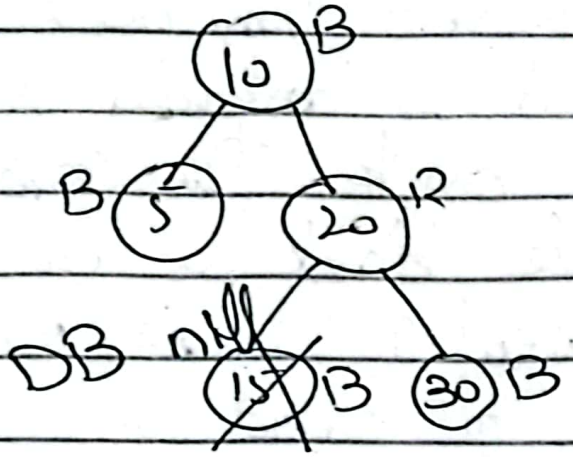
Path 1:- B→B
⇒ ② Black

Path 2:- B-R-B
⇒ ② Black

Path 3:- B-R
⇒ ① Black

* Case :-
If root is double Black (DB), just remove DB.

* Case 3 :-
If DB's sibling is black & both it's children are black.



Delete 15

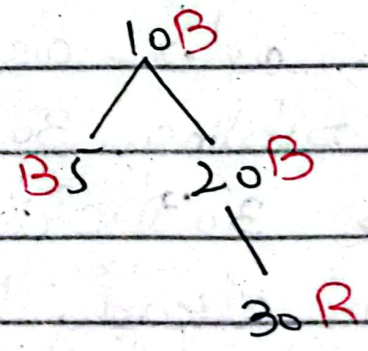
15 is also black and child of 15 is also black (nill is always black)

DB always check its sibling and color of sibling's children is also black. So after deleting 15 we, check it's sibling. 30 is also black, and also 30's children is black. Sibling is already black, So it not needs extra black.

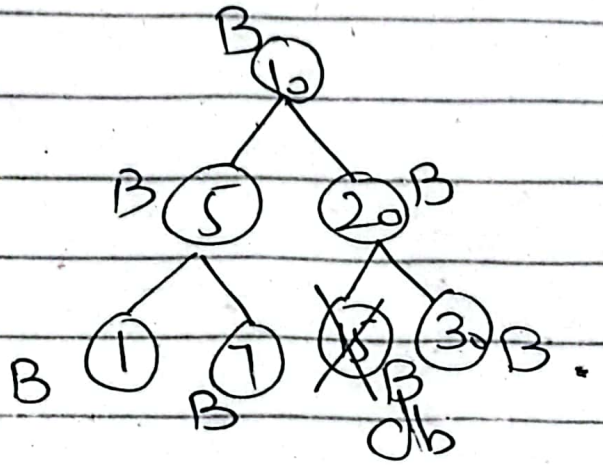
So just remove db, and the extra black gives to the parent of deleted node. and the sibling as black, so we give red to the sibling

* remove db
* add black to its parent (if parent isn't black
  * ⟶ if parent is red then add black
  otherwise if parent as already black then due to extra black, the parent will become double black so we don't do this if parent as black only give extra black to parent if parent as red.
* Make sibling red.

Sol. ⟹



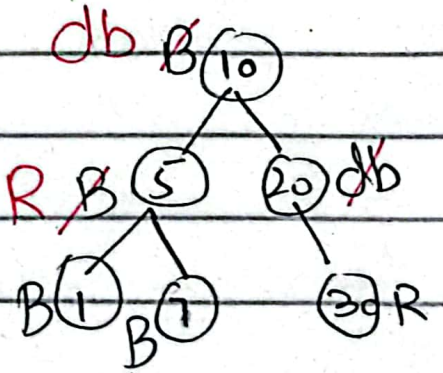
→ if still DB exists, apply other cases.



Delete 15

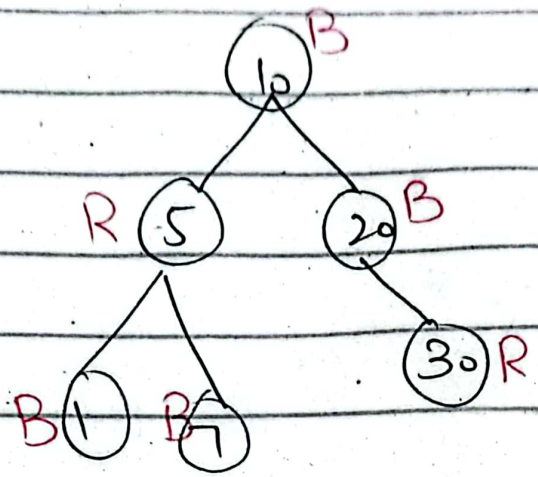
15 is already black and its children is also black (null).

Now parent is also, black so when give 15's black to its parent, the parent become db. and make the deleted node's sibling red.



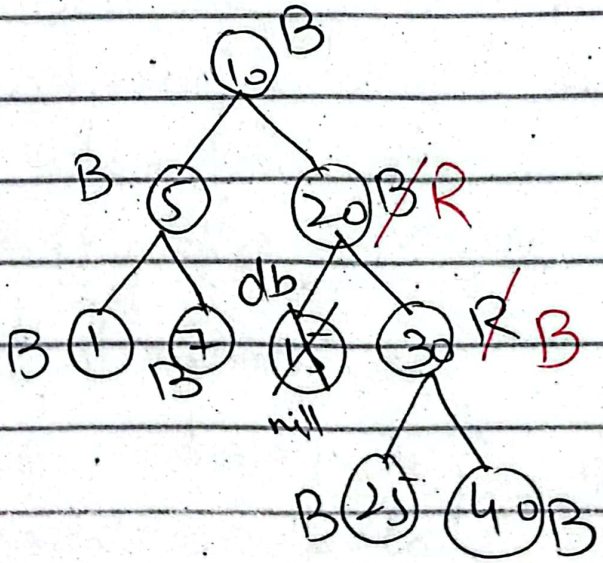
Now, remove db from 20, parent (10) become black and sibling's (5) become red.

Now, root is db. So simply remove db from root and do nothing simply.



Case 4:-

If DB's sibling is Red

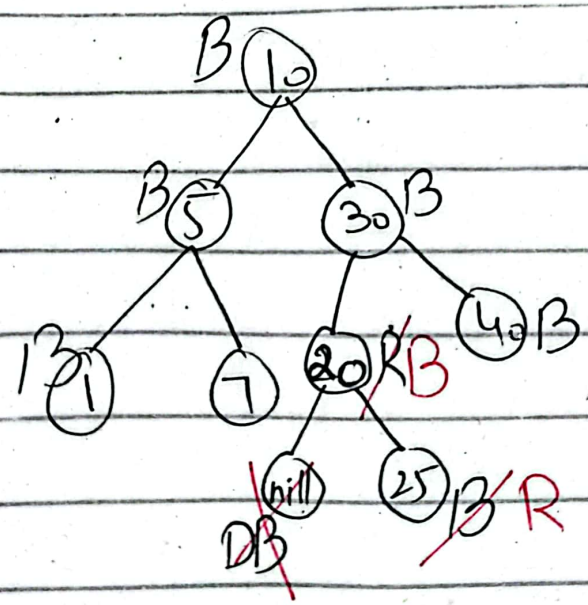


Delete 15

Now db after deletion.

Now, sibling is red but the children of sibling's is black.

Then; the parent of (15) ↑ will become red (20) and sibling become black (30) but still there is DB at the deleted node. So we rotate the parent in the direction of double Node (DB).



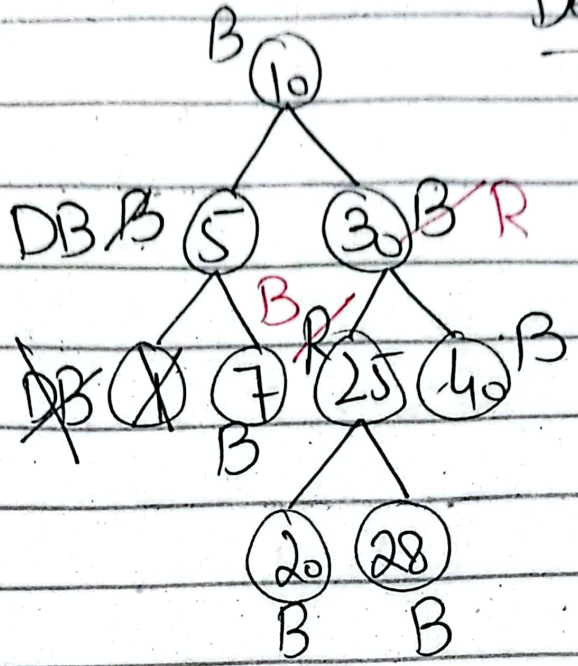
* swap colors of parent & it's sibling
* Rotate the parent in DB direction
* reapply cases.

Simply Now remove db and parent become red (extra black give to parent) and sibling become red.

DB's sibling is Black, sibling's child who is far from DB is black, but near child to DB is red.

Case 5 :- →

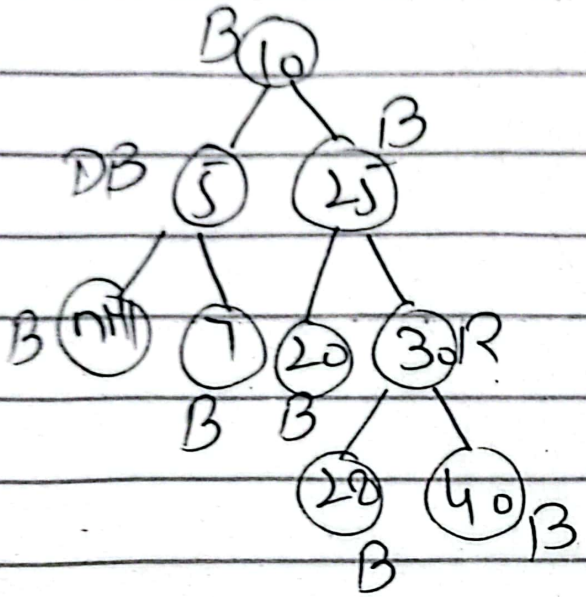
Delete 4



Now give extra black to parent (5)
Now parent(5) becomes DB and check the sibling of DB node (30). So sibling is also black, Now check the sibling's child who is far from DB (right child in this case of sibling) and check the near sibling's child (25) is Red.

* Swap the color of DB sibling (30) and the color of sibling's child near to DB as it red (25)

Now, rotate the DB's sibling in the opposite direction of DB's.



⇒ Apply Case #06

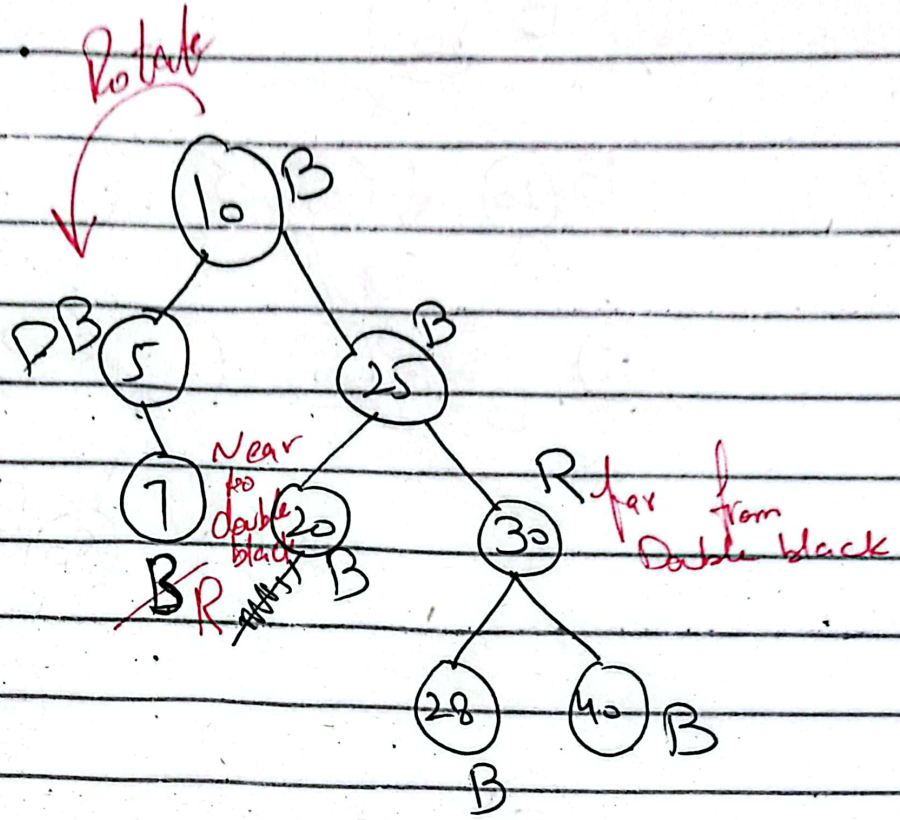
still have DB problem, so we have Case 6 to solve this problem

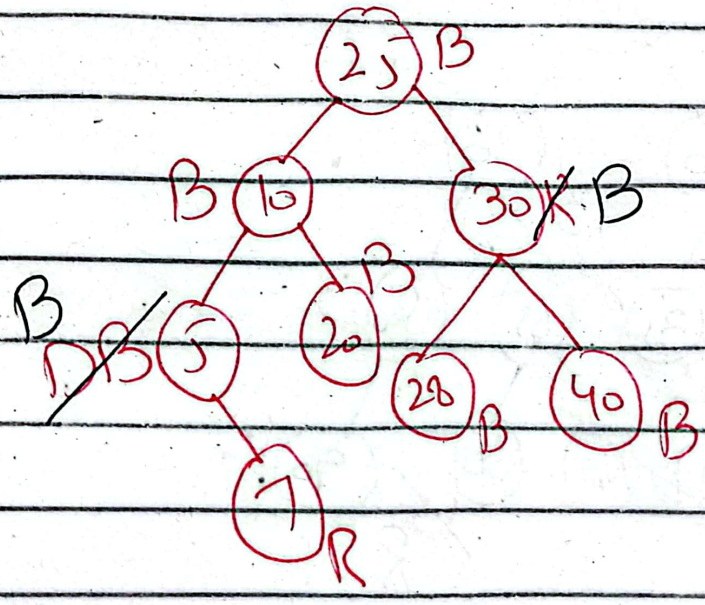
Case 6:-

DB's sibling is black, far child is red.

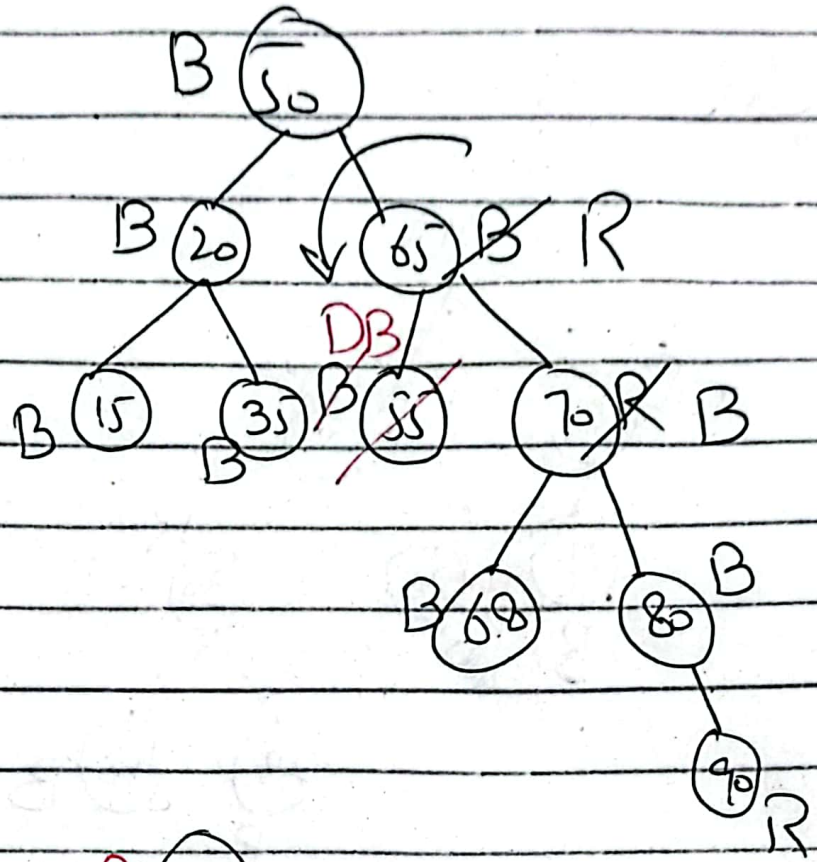
* swap color of parent & sibling
* Rotate parent in DB's direction.
* remove DB
* Change color of red child to

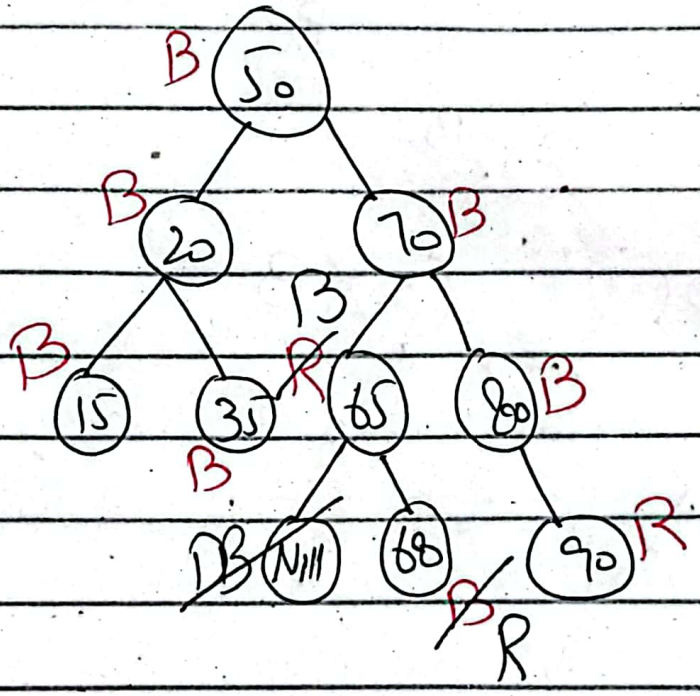
black.


Rotate
10
B
DB
5
25
B
7
Near to double black
20
B
R
B
R
30
far from Double black
28
B
40
B

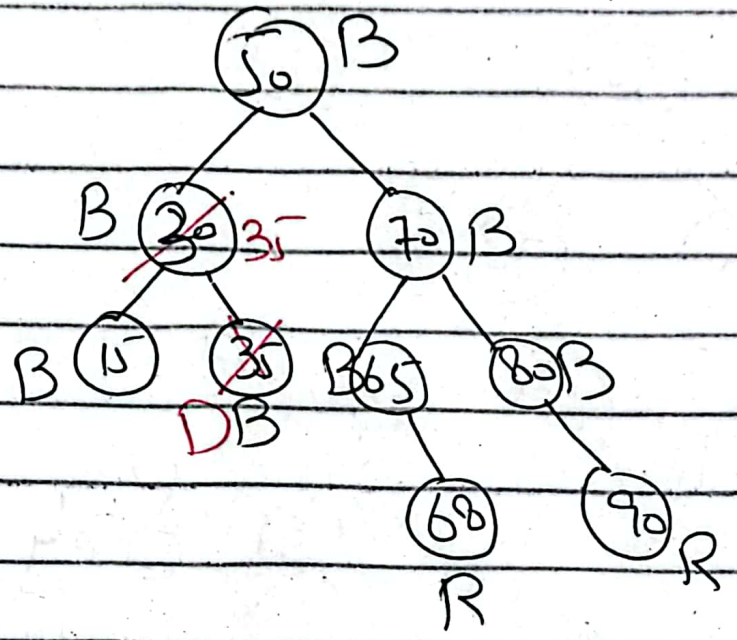

25
B
B
10
30
X
B
B
B
5
20
B
28
B
40
B
7
R

Delete 55

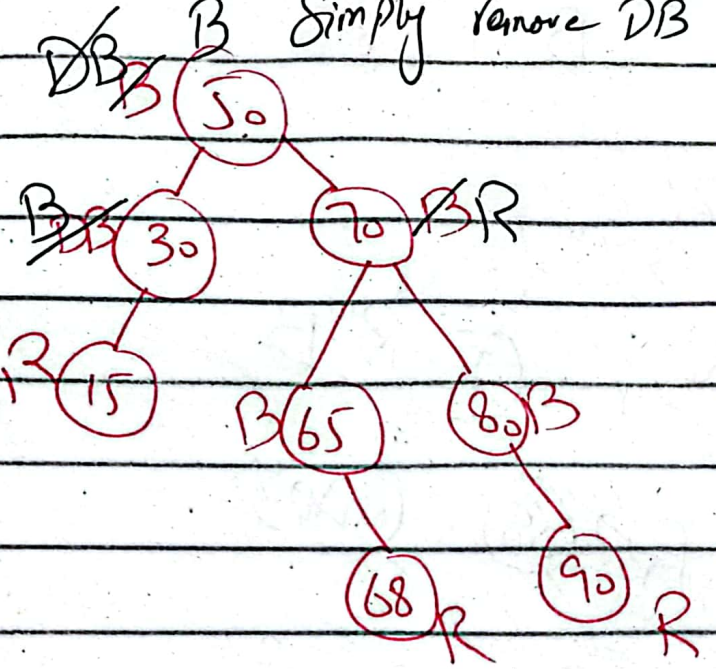

B
50
B
20
65
B
R
DB
B
15
35
B
55
70
R
B
B
68
80
B
90
R


B
50
B
20
70
B
B
B
15
35
R
65
80
B
B
DB
NIL
68
90
R
B
R

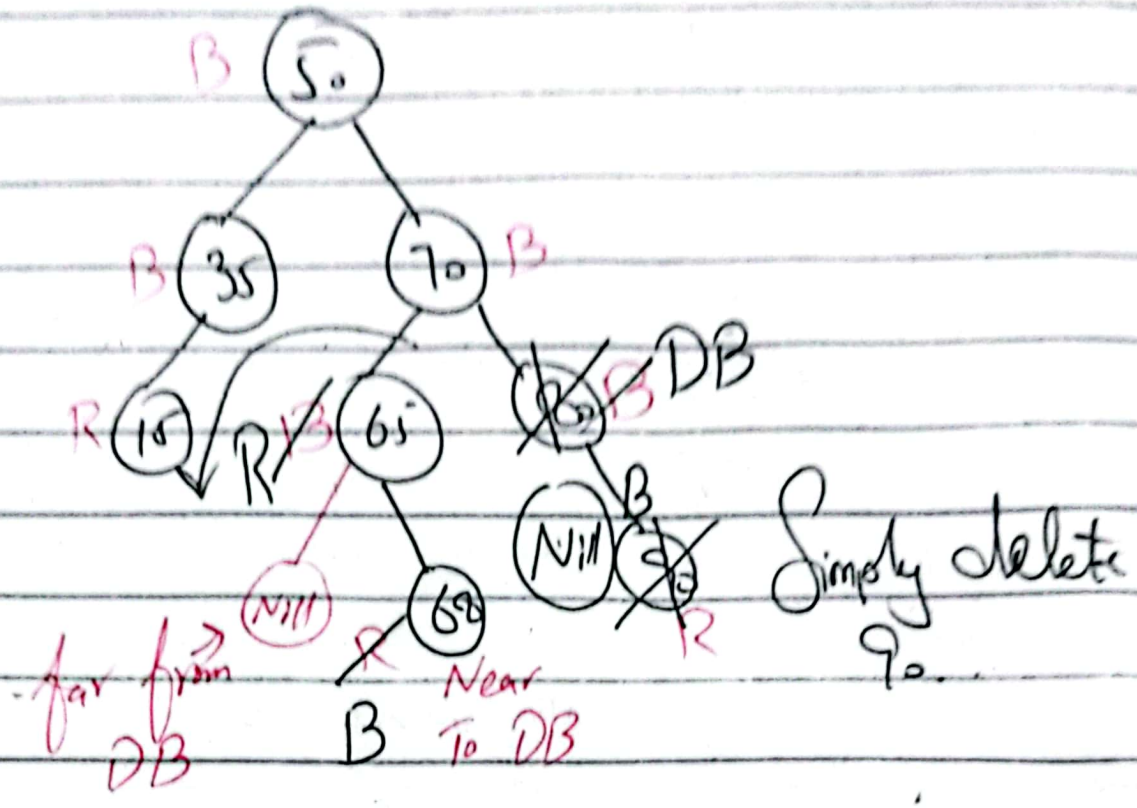
Delete 30 (Inorder Successor → 35)

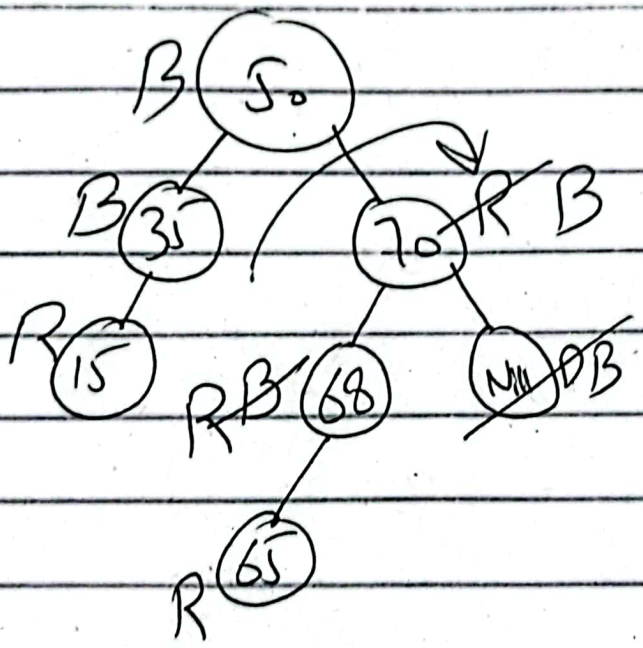


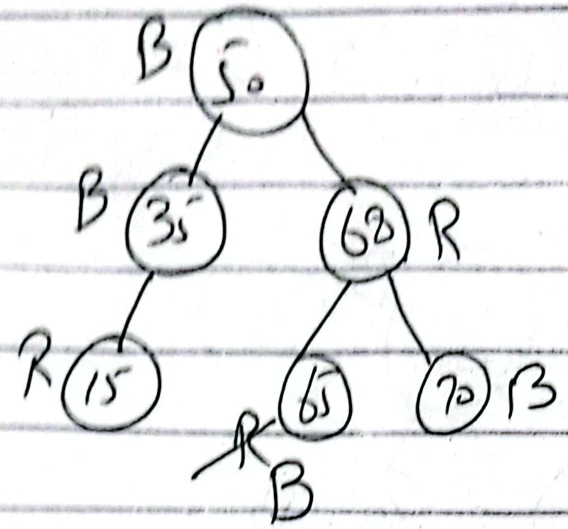
Simply remove DB



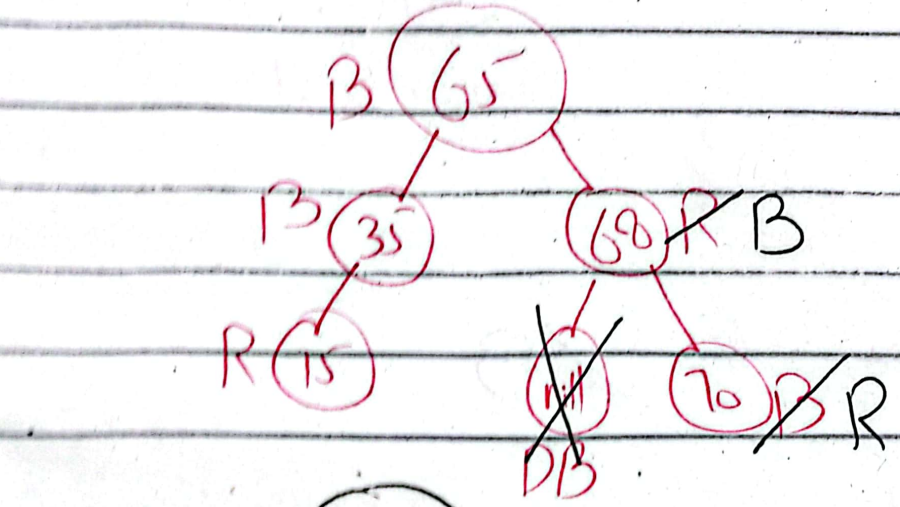
Delete 90. → Delete 80





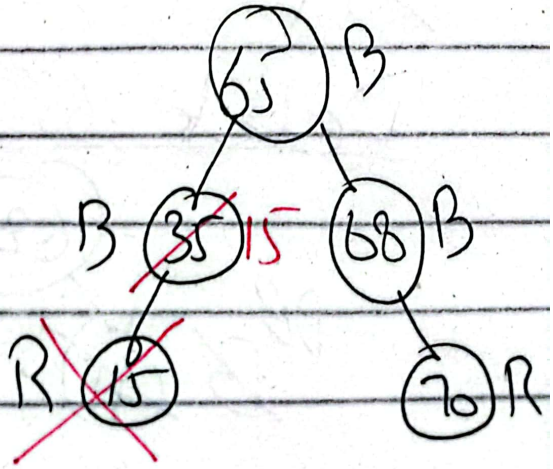


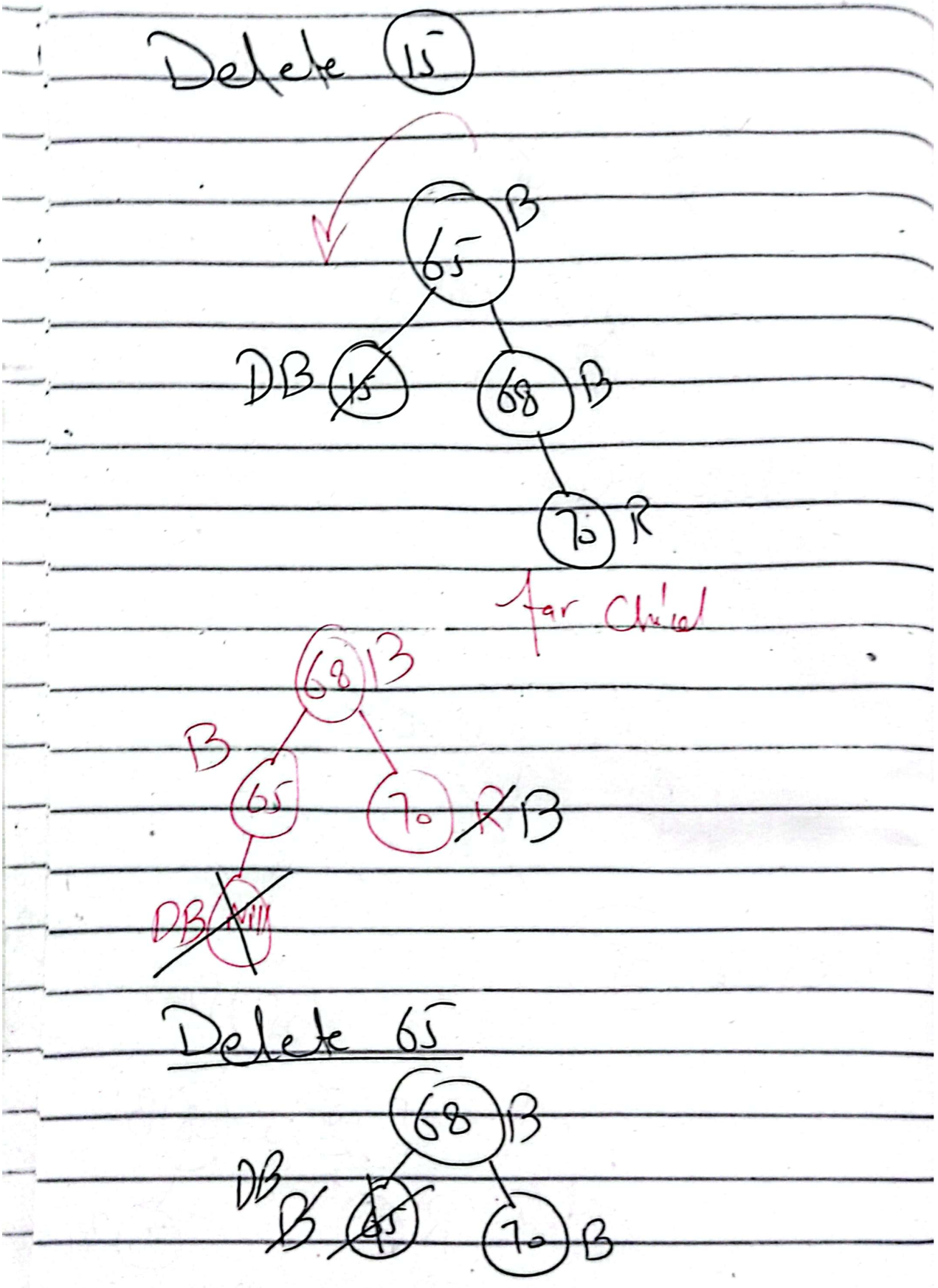
Delete 50



Delete 35

Simply delete it

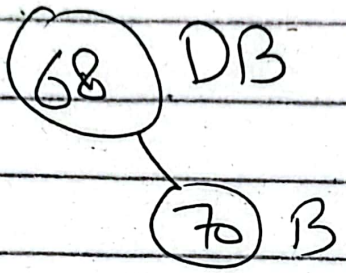


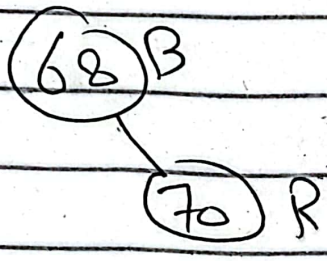

Delete 15
65 B
DB 15
68 B
70 R
far Child
68 B
B
65
70 R B
DB Null

Delete 65

68 B
DB
B
65
70 B

far child & near child both are black





Delete 68

